

Lab 8: A Low-Rank Update and a Forgetting Demo

Factor a 4×4 update, then train XOR then AND and watch the first task drop.

Work in a **Jupyter** notebook with **PyTorch**. Do notes **8.1–8.3** first. CPU is enough. No pretrained LLMs.

First time you implement this. The lecture notes this week already answered what the method is, why it exists, the architecture, the formula, and the tradeoffs. Read that first-time block before these exercises. This lab is the first time you **compute** it, not the first definition.

```
import torch
from torch import nn
import torch.nn.functional as F
```

1. Frozen W , low-rank BA

```
torch.manual_seed(0)
W = torch.randn(4, 4)
A = nn.Parameter(torch.randn(1, 4) * 0.1) # r = 1
B = nn.Parameter(torch.zeros(4, 1))
x = torch.randn(4)
```

Treat W as **frozen** (no `requires_grad` on a buffer).

1. Print $W @ x$ and $(W + B @ A) @ x$. They must match at init because B is zero.
2. Target $y = \text{torch.tensor}([1.0, -1.0, 0.5, 0.0])$. Train **only** A, B for 400 Adam steps, `lr=5e-2`, MSE on $(W + B @ A) @ x$ versus y . Print the residual $B @ A$ and its rank (`torch.linalg.matrix_rank`). Rank should be ≤ 1 .
3. Freeze A, B and try to fit the same y by training a copy of W (optional). The point of (2) is: a rank-1 bump can move the output without touching W .

2. Sequential XOR, then AND (forgetting)

Two-bit inputs, labels XOR, then the **same** net trained on AND.

```
bits = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
xor = torch.tensor([[0.], [1.], [1.], [0.]])
and_ = torch.tensor([[0.], [0.], [0.], [1.]])

class Tiny(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(2, 8), nn.Tanh(), nn.Linear(8, 1))
    def forward(self, z):
        return self.net(z)
```

1. Train `Tiny` on XOR to mean accuracy 1.0 (`BCEWithLogitsLoss`, a few hundred steps). Print predictions.
 2. Continue training the **same** weights on AND until AND accuracy is 1.0. Re-evaluate XOR. It should collapse toward chance or toward AND's pattern. That is catastrophic forgetting.
 3. **Replay:** from a fresh XOR-trained net, train on batches that are 50% AND rows and 50% XOR rows. Report XOR and AND accuracies. XOR should survive better than in (2).
-

3. Write-up

Four sentences: why `B=0` keeps the base map, what rank $r = 1$ cannot represent, what forgetting looked like on XOR, and why a per-task LoRA would not have required replay in (2).